Latent Merging: Dynamic and Reversible Composition of Large Language Models

JaeSeong Kim[1] and Suan Lee[1*]

Semyung University, Jecheon-si, Republic of Korea
{kjsqp1010, suanlee}@semyung.ac.kr

Abstract. Weight merging combines large language models by averaging parameters, but the resulting model is static and irreversible: no dynamic adaptation is possible at inference time. We propose Latent Merging, which composes models in hidden-representation space instead. Classic merging operators—LERP, SLERP, and RegMean—are unified under a single formalism and extended from weights to latent states, enabling dynamic, reversible, and layer-selective composition without modifying any model weights. We derive local second-order bounds on loss change under RMSNorm nonlinearity, yielding actionable guidance: merge at later layers and align LM heads. Experiments on two model pairs (Qwen2.5-7B / OpenThinker3-7B and Llama-3.1-8B / Nemotron-Nano-8B) across JudgeBench, HumanEval, and GSM8K show that latent merging consistently outperforms weight merging—achieving 96.9% JudgeBench win-rate versus 15.7% on the Qwen pair—while preserving representational structure (CKA > 0.8). Layer-wise analysis confirms the theoretical predictions and reveals that moderate mixing ratios in deeper layers yield the strongest results.

Keywords: latent merging · model merging · hidden representations · reversible composition · large language models · representation geometry

1 Introduction

Large language models (LLMs) are proliferating across domains, and the question of how to combine their complementary strengths has become practically important. Weight merging methods such as Model Soup [22] and Fisher-weighted merging [15] address this by synthesizing knowledge directly in parameter space, achieving a compelling combination of simplicity and effectiveness. Yet this simplicity comes at a cost: weight merging carries fundamental limitations that constrain its applicability.

Weight merging is both irreversible and global. Once parameters are merged, recovering the originals requires storing extra checkpoints and redistributing them. The merged model is static: it cannot adapt its composition to different

* Corresponding author

inputs or contexts at inference time. Selective combination of particular capabilities is not possible after the fact. More fundamentally, weight merging treats models as fixed artifacts to be rewritten offline, yielding a single outcome that cannot be adjusted without repeating the merge. These properties have practical consequences: merged models can exhibit repetition artifacts and capability collapse when the source models differ substantially [15].

We observe that Transformer hidden states offer a different substrate for composition [21]. Rather than fixed parameters, hidden representations are dynamic: they change with every input and evolve layer by layer as the model processes a sequence. Merging in this space means controlling ongoing computations rather than modifying stored functions. The result is reversible by construction, since no weights are changed. It is also conditional, since the merge can vary per token, per layer, or per context. And it is localized, since interventions at one layer need not propagate to others. These properties are structurally unavailable to weight-space methods.

We propose **Latent Merging**, a framework that generalizes weight-space merging operators to hidden representations. The three canonical operators, linear interpolation (LERP), spherical interpolation (SLERP), and regularized means (RegMean), are unified under a single operator view and extended from weights to latent states. We evaluate the framework on two model pairs spanning distinct training lineages: (i) **Qwen2.5-7B-Instruct** and **OpenThinker3-7B**, which share the same architecture and tokenizer within the Qwen2.5 family, and (ii) **Llama-3.1-8B-Instruct** [8] and **Llama-3.1-Nemotron-Nano-8B-v1** [2], a general-purpose and reasoning-specialized pair from the Llama 3.1 family. The inclusion of the second pair tests whether latent merging generalizes beyond a single model family while maintaining the shared-architecture prerequisite. Evaluation is conducted in a data-free setting on JudgeBench across four domains (Knowledge, Reasoning, Math, Coding), HumanEval for code generation, and GSM8K for mathematical reasoning.

The contributions of this work are as follows.

1. **Unified latent merging framework**: We extend weight-space merging operators (LERP, SLERP, RegMean) to hidden-state space, enabling dynamic, reversible, and layer-selective model composition without parameter modification.
2. **Theoretical analysis**: We derive local second-order bounds on loss change under RMSNorm nonlinearity, yielding practical guidance: merge at later layers and align LM heads to minimize interpolation error.
3. **Empirical validation and failure analysis:** We validate the theoretical predictions on two model pairs (Qwen2.5 and Llama 3.1 families) across JudgeBench (four domains), HumanEval, and GSM8K, with explicit cost analysis quantifying the 2× inference overhead. Beyond demonstrating consistent gains, we identify and characterize two systematic failure modes—final-layer collapse and alpha sensitivity—that are predicted by the second-order bounds and yield practical deployment guidelines.

The remainder of this paper is organized as follows: Section 2 reviews related work, Section 3 presents the framework and theory, Section 4 describes the experimental setup, Section 5 presents results, and Section 6 discusses limitations before concluding.

2 Related Work

Weight Merging. A substantial body of work composes models by directly averaging or combining their parameters. Wortsman et al. [22] showed that averaging the weights of multiple fine-tuned copies of the same base model improves out-of-distribution robustness, a technique they call Model Soup. Matena and Raffel [15] extended this idea by weighting each parameter by its Fisher information, improving stability when models diverge. Ilharco et al. [12] introduced task arithmetic, which treats the difference between a fine-tuned and a base model as a task vector that can be added or subtracted to compose capabilities. TIES-Merging [23] addresses sign conflicts and redundant parameters by trimming low-magnitude deltas and resolving disagreements through sign consensus before merging, reducing destructive interference. TaLoS [1] targets LoRA adapters specifically, using singular value decomposition to identify and merge task-relevant subspaces while discarding noise. Localize-and-Stitch [17] identifies the weight regions most responsible for each task and stitches them together selectively. Despite their diversity, all these methods share a fundamental property: they produce a single fixed model. The merge is performed offline and is irreversible without storing the original checkpoints. No dynamic adaptation to individual inputs or contexts is possible at inference time.

Latent Space Manipulation. A parallel line of work operates on hidden representations rather than weights. Dathathri et al. [5] introduced Plug and Play Language Models (PPLM), which steer generation by iteratively adjusting hidden states using gradients from an external attribute classifier. Subramani et al. [19] showed that latent steering vectors can be extracted from pretrained models and added to hidden states to control sentence style and semantics without any fine-tuning. Turner et al. [20] and Panickssery et al. [16] demonstrated that contrastive activation addition, computing the difference between activations for contrasting prompts, produces reliable steering directions across a range of behaviors. Diallo et al. [6] applied similar ideas to style transfer, extracting style vectors that shift generation toward a target register. In vision, GAN research has long shown that interpolating latent codes produces semantically smooth transitions between concepts. Collectively, these works establish that hidden representations are a controllable and compositional substrate, offering properties that weight-space methods structurally cannot provide.

Ensembling and Distillation. Knowledge distillation [10] compresses ensemble behavior into a single model by training a student to match the soft targets of a teacher, recovering single-model inference cost at the price of a training

phase. Mixture-of-Experts architectures [7] achieve conditional routing at scale by directing each token to a subset of specialized expert modules, but require architectural changes and joint training from the outset.

Relation to Our Work. Weight merging is simple and requires no additional inference cost, but it is irreversible, static, and uncontrollable: the merged model cannot adapt its composition to different inputs. Latent manipulation methods are controllable and reversible, but they focus on attribute steering within a single model rather than composing distinct models. Our Latent Merging bridges these lines of work. It applies generalized merging operators directly in hidden-state space, combining the simplicity of weight merging with the controllability of latent manipulation.

3 Methodology

Existing weight merging techniques can be understood as special cases of a single, more general composition framework. Rather than treating linear interpolation or spherical interpolation as independent heuristics, we cast them under a unified operator formalism. This abstraction clarifies the relationships among prior methods and provides a natural pathway to extend merging from parameter space to latent representation space, enabling dynamic and reversible model composition.

3.1 Generalized Merging Operator

We begin by formalizing weight merging as a generalized operator that subsumes a wide range of existing techniques. Given two models A and B with parameters θ_A and θ_B, a merging operation can be expressed as:

$$\theta' = \mathcal{M}(\theta_A, \theta_B; \alpha, G) \tag{1}$$

where $\alpha \in [0, 1]$ denotes the mixing coefficient, G specifies the space in which the merge is performed, and $\mathcal{M}$ is the merging operator defined with respect to G.

Three canonical instantiations cover the methods we study. **LERP** sets G to Euclidean space and computes $\theta' = (1 - \alpha)\theta_A + \alpha\theta_B$, the standard convex combination. **SLERP** sets G to the unit hypersphere, interpolating along the geodesic arc between unit-normalized parameters to preserve angular structure. **RegMean** [13] sets G to a weighted inner-product space, computing a regularized mean that accounts for activation statistics and reduces interference between task-specific directions.

Viewed this way, weight merging is not a collection of ad-hoc procedures but an instantiation of a general family of parameter-space composition rules. This abstraction is what makes the extension to latent space natural.

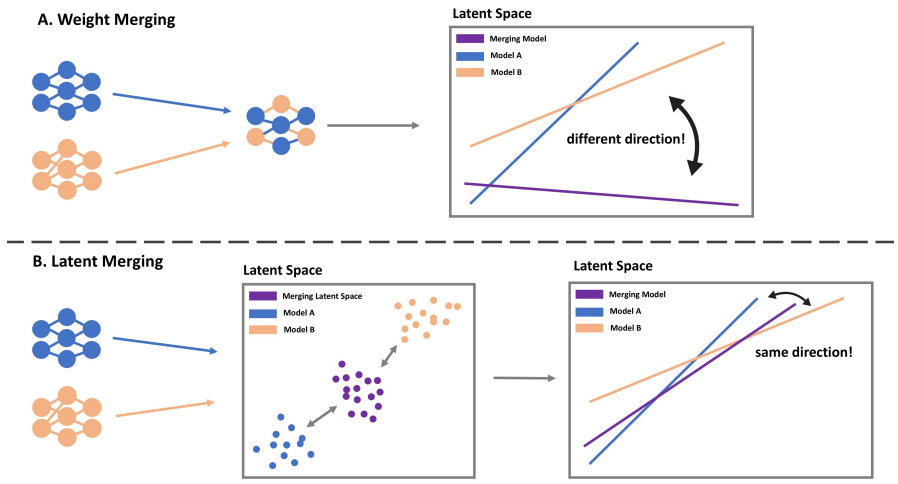

Fig. 1. Overview of Latent Merging. (A) Weight merging composes parameters offline, producing a single static model. (B) Latent merging dynamically composes hidden states at inference time, enabling reversible and layer-selective control without modifying any weights.

3.2 Latent Merging Framework

We extend the notion of merging from parameters to hidden representations. Let h_A and h_B denote the latent states produced by two models at a given layer. We define latent merging as:

$$h'_\alpha = \mathcal{M}(h_A, h_B; \alpha, G) \tag{2}$$

where $\mathcal{M}$ is the same operator family as in Eq. (1) and $\alpha \in [0, 1]$ controls the mixing ratio. This formulation composes internal representations rather than model parameters, preserving the full operator family while avoiding the rigidity of weight-level modifications.

A crucial factor lies in how merged representations map to outputs. Modern LLMs apply a nonlinear pre-normalization $r(h)$ (e.g., RMSNorm [24]) before the final linear projection, yielding logits $g_A(h) = W_A r(h) + b_A$. This architectural detail changes the theoretical landscape: the loss under a merged state is no longer a direct Jensen-type interpolation of the source losses. Instead it includes correction terms that reflect the curvature of the normalization and potential mismatches across LM heads. These considerations explain why merging at later layers tends to be more robust and why head alignment often plays a critical role, as we formalize next.

At inference time, both source models run their forward passes independently up to the chosen merge layer. The hidden states h_A and h_B are combined via Eq. (2), and the merged state h'_α is passed through model A's RMSNorm and LM head to produce the output distribution. No weights are modified; the merge is fully reversible by changing α or the merge layer.

3.3 Theoretical Analysis

We study latent merging in the presence of a nonlinear pre-head $r\colon \mathbb{R}^d \to \mathbb{R}^d$ (e.g., RMSNorm) followed by a linear LM head. For model A, let $g_A(h) = W_A r(h) + b_A$, and define $z_A^{(A)} = g_A(h_A)$ and $z_B^{(A)} = g_A(h_B)$. We evaluate with a convex, β-smooth logit-based loss ℓ (e.g., cross-entropy).

Assumption 1 (Local smoothness). r is C^2 on a neighborhood of the segment $[h_A, h_B]$ and its Jacobian is K_r-Lipschitz there. Equivalently, the Jacobian of g_A is K_g-Lipschitz with $K_g \le \|W_A\|_{\mathrm{op}} K_r$.

Theorem 1 (Local second-order bound). *Under Assumption 1, for $h'_\alpha = (1-\alpha)h_A + \alpha h_B$ with $\alpha \in [0,1]$:*

$$\begin{aligned}\ell\big(g_A(h'_\alpha)\big) \le{}& (1-\alpha)\,\ell\Big(z_A^{(A)}\Big) + \alpha\,\ell\Big(z_B^{(A)}\Big)\\ &+ \frac{GK_g}{2}\,\alpha(1-\alpha)\,\|h_B - h_A\|^2\\ &+ \frac{\beta K_g^2}{8}\,\alpha^2(1-\alpha)^2\,\|h_B - h_A\|^4 \qquad (3)\end{aligned}$$

where $G = \sup_u \|\nabla \ell(u)\|$ along the interpolated logit segment. The proof follows from a Taylor expansion of g_A around h_A and β-smoothness of ℓ.

Practical Diagnostic. Theorem 1 is not a performance prediction but a diagnostic tool: it identifies which structural factors drive interpolation error. The correction terms in Eq. (3) scale as $O(\alpha(1-\alpha)\|h_B - h_A\|^2)$ plus a quartic term, yielding three actionable predictions.

1. **Merge at later layers.** In deeper layers, representations from models trained on related data tend to converge, so $\|h_B - h_A\|$ is smaller and the correction terms shrink.
2. **Keep mixing ratio moderate.** The factor $\alpha(1-\alpha)$ is maximized at $\alpha = 0.5$ and decreases toward the endpoints; the quartic term $\alpha^2(1-\alpha)^2$ peaks at $\alpha = 0.5$ as well. In practice, $\alpha \approx 0.5$–0.75 balances capability blending against interpolation error.
3. **Align LM heads.** When model B's head g_B differs from g_A, an additional head-mismatch term $\alpha G \Delta_{\mathrm{head}}(B{\to}A)$ enters the bound, where $\Delta_{\mathrm{head}}(B{\to}A) = \|(W_A - W_B)r(h_B) + (b_A - b_B)\|$. Minimizing this mismatch directly reduces the bound.

We empirically validate these predictions in Section 5.

Claim (Linear limit). If r is the identity, then $K_g = 0$, the curvature terms vanish, and Eq. (3) reduces to the exact Jensen interpolation $(1-\alpha)\ell(z_A^{(A)}) + \alpha\ell(z_B^{(A)})$.

3.4 Merging Operators

The same three operators from Section 3.1 apply directly to latent states. We summarize their behavior in the hidden-state setting.

LERP computes $h' = (1-\alpha)h_A + \alpha h_B$, the standard linear interpolation. It is the simplest operator and corresponds to the $K_g = 0$ regime of Theorem 1 when representations are well-aligned.

SLERP interpolates along the geodesic arc on the unit hypersphere. For unit-normalized states $\hat{h}_A$ and $\hat{h}_B$ with angle $\Omega = \arccos(\hat{h}_A \cdot \hat{h}_B)$:

$$h' = \frac{\sin((1-\alpha)\Omega)}{\sin \Omega}\,\hat{h}_A + \frac{\sin(\alpha\Omega)}{\sin \Omega}\,\hat{h}_B \tag{4}$$

SLERP preserves the unit-sphere structure of normalized representations, which is particularly relevant after RMSNorm normalization.

RegMean [13] computes a regularized mean weighted by activation statistics, reducing interference between task-specific directions. Applied to latent states, it weights each model's contribution by the inverse of its activation covariance, favoring directions where each model is more confident.

All three operators are applied at a chosen merge layer l^*, with α and l^* treated as hyperparameters. The theory in Section 3.3 predicts that larger l^* (later layers) and moderate α will perform best, a prediction we test directly in Section 5.

4 Experimental Setup

4.1 Models

We experiment with two model pairs, each consisting of a general-purpose base and a reasoning-specialized derivative that share the same architecture and tokenizer.

Pair 1: Qwen2.5 family (7B). **Qwen2.5-7B-Instruct** [11] is a general-purpose instruction-following model trained on a broad mixture of supervised fine-tuning data. **OpenThinker3-7B** [9] is a reasoning-specialized derivative obtained by further fine-tuning Qwen2.5-7B-Instruct on chain-of-thought and problem-solving data. Both models share the same architecture (28 transformer layers, grouped-query attention, RMSNorm), the same tokenizer, and the same vocabulary.

Pair 2: Llama 3.1 family (8B). **Llama-3.1-8B-Instruct** [8] is Meta's general-purpose instruction-tuned model built on the Llama 3.1 architecture (32 transformer layers, grouped-query attention, RMSNorm). **Llama-3.1-Nemotron-Nano-8B-v1** [2] is NVIDIA's reasoning-enhanced derivative, further fine-tuned from Llama-3.1-8B-Instruct on reasoning and problem-solving data. The two models share the same architecture, tokenizer, and vocabulary.

Design rationale. Both pairs follow a *base* → *specialist* pattern: a general-capability model is paired with a reasoning-focused fine-tune derived from it. This controlled setup isolates the effect of merging operators from confounds introduced by architectural or tokenizer mismatches. By evaluating across two distinct model families (Qwen2.5 and Llama 3.1) that differ in hidden dimension, layer count, and pretraining data, we test whether the benefits of latent merging generalize beyond a single lineage. The shared-architecture prerequisite remains: hidden states from both models in a pair must occupy the same dimensional space to be interpolated without learned projections. Extending latent merging to cross-architecture settings is left to future work.

4.2 Evaluation Protocol

JudgeBench. Our primary evaluation uses JudgeBench [18], a pairwise comparison framework that measures response quality across four domains: Knowledge (MMLU-Pro), Reasoning (LiveBench), Math (LiveBench), and Coding (LiveCodeBench). We use **GPT-4o** (`gpt-4o-2024-05-13`) as the judge model. Results are reported as win-rates: the percentage of pairwise comparisons in which the merged model's response is preferred over the OpenThinker3 no-merge baseline.

Order randomization. Position bias is a known confound in pairwise LLM evaluation, where judges tend to favor whichever response appears first. To mitigate this, for each pair we evaluate both orderings (A vs. B and B vs. A) and aggregate results per the JudgeBench protocol [18]. This order randomization ensures that win-rate differences reflect genuine quality differences rather than presentation artifacts.

HumanEval. We evaluate code generation using HumanEval [3], a benchmark of 164 Python programming problems. Each problem provides a function signature and docstring; models must complete the implementation. We use pass@1 accuracy: a solution is correct if it passes all hidden unit tests when executed in a sandboxed subprocess. This provides a complementary, execution-based signal to JudgeBench.

GSM8K. We evaluate mathematical reasoning using a fixed 300-question random subset of GSM8K [4] (seed 42). Models generate free-form solutions and accuracy is measured by exact match of the final numerical answer.

4.3 Baselines

We compare latent merging against three categories of baselines.

No-merge (standalone). Each source model is evaluated independently: Qwen2.5-7B-Instruct and OpenThinker3-7B run without any merging. These results establish the performance ceiling of the individual components and serve as the reference point for all pairwise comparisons.

Weight merging. We apply LERP, SLERP, and RegMean directly to model parameters, producing a single merged checkpoint offline. Weight merging is

irreversible: once parameters are combined, the originals cannot be recovered without storing separate checkpoints. This is the standard baseline in the model merging literature.

Latent merging (ours). LERP, SLERP, and RegMean are applied to hidden states at a chosen layer during the forward pass. No weights are modified; the merge is dynamic and reversible.

4.4 Merging Configuration

Layer selection. We evaluate merging at seven checkpoints across the 28-layer network: L0, L5, L10, L15, L20, L25, and L27. This spacing covers early (syntactic), middle (semantic), and late (task-specific) layers, enabling a systematic layer-wise analysis.

Mixing ratios. For each layer and operator, we sweep $\alpha \in \{0.25, 0.50, 0.75\}$, where $\alpha = 0$ recovers the first source model and $\alpha = 1$ recovers the second. This gives $7 \times 3 = 21$ configurations per operator.

Data-free setting. No fine-tuning or training data is required at merge time. All configurations are evaluated directly on JudgeBench without any task-specific adaptation.

Configuration selection. For each method (weight merging and latent merging, per operator), we report the best configuration selected by overall JudgeBench win-rate. This mirrors the practical use case where a practitioner tunes the merge on a held-out validation set.

4.5 Cost Measurement

To characterize the inference overhead of latent merging, we measure the following quantities on a single NVIDIA A100 GPU with batch size 1.

- **Wall-clock latency** (seconds per query): end-to-end time from prompt submission to final token.
- **Time-to-first-token (TTFT)**: latency until the first output token is generated, reflecting prefill cost.
- **Tokens per second (TPS)**: decode throughput.
- **Peak GPU memory** (MB): measured via `torch.cuda.max_memory_allocated()` to capture the additional memory required to hold activations from both models simultaneously.
- **Forward passes per query**: latent merging requires two forward passes (one per source model) per token.

These measurements allow a direct cost comparison between latent merging and the no-merge baselines, making the 2× compute overhead explicit and quantified.

Table 1. Main results across two model pairs. JudgeBench: win-rates (%) versus the respective specialist baseline (OpenThinker3 for Qwen2.5; Nemotron-Nano for Llama 3.1) across Knowledge (K), Reasoning (R), Math (M), Coding (C), and Overall (Avg). HEval: HumanEval pass@1 (%) on 164 code-generation problems [3]. GSM8K: accuracy (%) on a fixed 300-question random subset. Best per column per pair in **bold**.

		JudgeBench Win-Rate (%)					Accuracy (%)		Cost	
Space	**Method**	**K**	**R**	**M**	**C**	**Avg**	**HEval**	**GSM8K**	**Lat. (s)**	**VRAM**
Pair 1: Qwen2.5-7B-Instruct & OpenThinker3-7B										
Weight	LERP	19.7	5.9	23.1	3.4	15.7	55.5	25.7	2.82	29.1
	SLERP	2.9	3.0	7.7	1.8	3.4	66.5	57.3	2.82	29.1
	RegMean	18.5	6.0	24.2	1.7	14.9	55.5	25.7	2.82	29.1
Latent	LERP	82.1	75.9	83.5	85.7	81.6	**87.0**	91.0	5.52	29.1
	SLERP	93.4	91.0	**100.0**	**100.0**	94.3	76.8	90.7	5.52	29.1
	RegMean	**96.0**	**98.9**	96.0	**100.0**	**96.9**	80.0	**93.0**	5.52	29.1
Pair 2: Llama-3.1-8B-Instruct & Nemotron-Nano-8B-v1										
Weight	LERP	74.7	**58.2**	53.9	62.1	67.8	54.9	80.3	4.23	15.0
	SLERP	79.9	49.4	58.3	55.2	69.1	58.5	88.0	3.96	15.0
	RegMean	74.2	54.4	56.2	61.4	67.2	54.9	80.3	4.14	15.0
Latent	LERP	**84.0**	53.7	**61.8**	**79.0**	**75.9**	63.4	87.0	8.16	30.0
	SLERP	80.3	51.6	41.1	74.2	70.0	**67.1**	**90.0**	11.05	30.0
	RegMean	63.9	55.8	51.9	65.6	61.3	65.9	87.0	6.72	30.0

5 Results

5.1 Main Results

Table 1 reports win-rates on JudgeBench, pass@1 on HumanEval and accuracy on GSM8K, alongside wall-clock latency and peak VRAM for every method.

Overall findings. Latent merging consistently outperforms weight merging across both model pairs. For the **Qwen pair**, the best latent configuration (RegMean at L25, α=0.5) achieves 96.9% overall JudgeBench win-rate versus 15.7% for the best weight-merging variant, with all three latent operators exceeding 80% while all weight operators stay below 20%. On HumanEval and GSM8K, all latent operators surpass their weight-merging counterparts; notably, Latent-LERP (87.0%) exceeds both standalone models on HumanEval, and weight merging collapses to 25.7% on GSM8K.

For the **Llama pair**, the directional advantage holds but with a narrower gap, as weight merging does not catastrophically fail. On JudgeBench, latent-LERP leads at 75.9% (+8.1pp over weight-LERP), while latent-RegMean (61.3%) underperforms weight-RegMean (67.2%), indicating operator-dependent behavior. On HumanEval and GSM8K, all latent operators again surpass their weight

counterparts. The narrower gap is expected: Nemotron-Nano is a closer derivative of its base than OpenThinker3, making weight-space interpolation less destructive. Overall, latent merging wins 8/9 operator-wise comparisons across both pairs and all benchmarks.

Operator ranking in latent space. Among latent operators on the Qwen pair, RegMean ranks highest (96.9%), followed by SLERP (94.3%) and LERP (81.6%). RegMean's Fisher-weighted averaging accounts for feature importance when combining activations, while SLERP's norm-preserving interpolation prevents the systematic shrinkage of linear interpolation. Lemma 1 bounds the SLERP–LERP perturbation as $\delta_\alpha^{\text{slerp}} \leq c\,\alpha(1-\alpha)\,\theta^2 \max\{\|\mathbf{h}_A\|, \|\mathbf{h}_B\|\}$, confirming convergence when hidden states are well-aligned ($\theta \to 0$), consistent with our CKA analysis (Section 5.2).

Cost. Latent merging incurs ≈1.96× latency overhead (two concurrent forward passes per token). For the Qwen pair, VRAM increase is negligible (+18 MB); for the Llama pair, peak VRAM doubles (30.0 GB vs. 15.0 GB) since both source models must reside in memory. In exchange, latent merging preserves full reversibility and layer-selective control. All win-rates use GPT-4o as judge with order randomization to control for position bias.

5.2 Representational Similarity Analysis

To understand *why* latent merging outperforms weight merging, we measure three geometric properties of the merged hidden states: **Midness** (cosine similarity to the midpoint between source representations), **Arc Deviation** (angular deviation from the geodesic midpoint, in radians), and **CKA** (Centered Kernel Alignment [14] between the merged model and each source, averaged across layers). Table 2 reports these metrics for the best configuration of each operator.

Table 2. Representational similarity metrics for latent vs. weight merging (best configuration per operator). Midness: cosine similarity to the interpolation midpoint (↑). Arc Dev: angular deviation from geodesic midpoint in radians (↓). CKA: Centered Kernel Alignment with source models (↑).

	Midness (↑)		Arc Dev (↓)		CKA (↑)	
Method	Latent	Weight	Latent	Weight	Latent	Weight
SLERP	**0.80**	0.61	**0.12**	0.19	**0.89**	0.35
LERP	**0.77**	0.64	**0.04**	0.32	**0.83**	0.32
RegMean	**0.78**	0.64	**0.02**	0.16	**0.83**	0.35

Table 3. LERP win-rates (%) by merge layer and mixing ratio α. K = Knowledge, R = Reasoning, M = Math, C = Coding. **Bold** marks the best α per layer per domain. † marks catastrophic collapse (<10% across most domains).

	Qwen2.5-7B-Instruct												OpenThinker3-7B											
	K			R			M			C			K			R			M			C		
Layer	.25	.50	.75	.25	.50	.75	.25	.50	.75	.25	.50	.75	.25	.50	.75	.25	.50	.75	.25	.50	.75	.25	.50	.75
L5	6.7	10.0	8.3	15.8	17.8	14.1	28.5	22.0	25.7	22.3	26.5	**37.6**	47.1	41.7	45.7	86.6	87.1	**87.2**	58.3	54.2	**71.1**	89.0	88.2	**90.4**
L10	11.1	8.3	7.4	22.9	10.6	19.5	16.9	13.5	**27.0**	27.3	**33.6**	32.5	46.5	36.9	42.8	86.4	83.6	**85.5**	**71.6**	44.5	60.6	86.8	**97.2**	87.9
L15	13.3	8.6	9.5	10.5	9.5	**15.3**	19.1	28.5	**29.6**	17.5	**18.0**	17.4	46.4	44.5	40.7	79.7	**86.0**	74.5	64.3	56.3	**59.9**	71.7	83.7	**86.8**
L20	11.9	16.0	**17.6**	9.1	7.2	0.0	15.6	**28.3**	16.7	26.4	**29.6**	27.0	60.1	64.2	**78.2**	71.5	65.8	**69.8**	63.1	**77.1**	63.6	85.7	94.2	**96.2**
L25	12.6	**15.4**	10.8	6.4	3.2	0.0	14.9	**14.8**	15.0	**28.6**	23.0	10.0	65.4	76.7	**82.1**	**78.4**	77.6	75.9	73.9	82.0	**83.5**	**91.2**	85.5	85.7
L27†	39.0	6.4	1.6	14.1	0.0	0.0	23.7	7.7	7.4	34.5	0.0	0.0	95.7	72.7	49.5	90.2	53.0	20.9	86.8	77.1	66.9	87.5	29.3	26.8

Latent merging preserves representational structure. CKA exceeds 0.80 for all three latent operators, compared to 0.32–0.35 for their weight-space counterparts. This gap confirms that hidden states remain semantically coherent after latent interpolation: the merged model's internal representations stay close to those of both source models across all layers. Weight merging, by contrast, produces representations that diverge substantially from either source, consistent with the parameter interference observed in prior work.

Midness confirms balanced interpolation. Midness values of 0.77–0.80 for latent merging indicate that the merged hidden states sit genuinely between the two sources rather than collapsing toward one. Weight merging achieves only 0.61–0.64, suggesting the merged parameters drift off the interpolation midpoint. Arc Deviation tells the same story: latent merging deviates by at most 0.12 radians from the geodesic midpoint, while weight merging deviates by up to 0.32 radians.

Connection to Theorem 1. These results validate the Practical Diagnostic from Section 3.3: the correction terms in Theorem 1 scale as $O(\alpha(1-\alpha)\|h_B - h_A\|^2)$, so smaller $\|h_B - h_A\|$ (reflected in high CKA and Midness) corresponds to tighter bounds. Latent merging at later layers, where representations from related models are most aligned, achieves the smallest deviation and the highest CKA, directly explaining why later-layer interventions yield the best performance in Table 1. Weight merging bypasses this alignment entirely, operating in parameter space where the analogous distance is large and uncontrolled.

5.3 Layer and Ratio Analysis

Table 3 reports LERP win-rates across six merge layers and three mixing ratios for both source models. LERP is the best-performing operator overall and provides the clearest signal for layer-wise trends.

Later layers dominate. Across both model families, L20–L25 consistently outperforms L5–L15. This aligns with the theoretical prediction from Theorem 1: deeper layers encode semantically richer representations, so $\|h_B - h_A\|$ is smaller and the interpolation error bound tightens. For OpenThinker3, Knowledge at L25 reaches 82.1% at α=0.75, and Coding at L20 reaches 96.2% at α=0.75.

Higher ratios are generally better and stable. α=0.75 is optimal in the majority of L20–L25 configurations. Crucially, higher mixing ratios do not induce instability in these layers; they reliably enhance transfer. This is consistent with the $\alpha(1-\alpha)$ factor in Eq. (3): when $\|h_B - h_A\|$ is small (as in later layers), even large α keeps the correction terms bounded.

Operator-specific nuance. LERP is reliably effective in deeper layers, particularly for OpenThinker3. SLERP shows comparative strength in very early layers (Qwen2.5 L5), where the angular correction is more meaningful. RegMean peaks in middle-to-late layers and attains several best-in-row scores. Despite these nuances, the global pattern is consistent: later layers with higher ratios are most favorable.

5.4 Failure Modes

Two systematic failure modes emerge from the layer-wise analysis. We report them explicitly rather than omitting them, as they carry practical and theoretical significance.

L27 (Final Layer) Collapse. The final Transformer block (L27) consistently exhibits catastrophic performance degradation across all operators and mixing ratios. For LERP on Qwen2.5 at α=0.75: Knowledge=1.64%, Reasoning=0.00%, Math=7.35%, Coding=0.00%. At α=0.50 the collapse persists: Knowledge=6.35%, Reasoning=0.00%, Math=7.72%, Coding=0.00%. This pattern holds across all three operators (LERP, SLERP, RegMean) and is not an artifact of a single configuration.

OpenThinker3 shows a milder but structurally identical pattern: Coding at L27 drops to 29.3% and 26.8% at α=0.50 and α=0.75 respectively, compared to 85.5% and 85.7% at L25. The degradation is sharpest in Reasoning and Coding, the domains most sensitive to task-routing precision.

We attribute this to the role of the final Transformer block in encoding task-specific features at high precision. The LM head relies on these final-layer representations to route predictions; merging at L27 destroys the critical task-routing information before it reaches the head. Formally, this corresponds to large $\|h_B - h_A\|$ at the final layer, which inflates the K_g-dependent correction terms in Theorem 1 and pushes the bound well above the source-model losses.

Alpha Sensitivity. Performance varies significantly with α across layers. While α=0.75 is generally optimal in deeper layers (L20–L25), the same ratio causes instability at L27. For Qwen2.5 LERP, Knowledge drops from 39.0% at α=0.25 to 1.6% at α=0.75 at L27, a 37-point collapse driven purely by increasing the mixing ratio. This sensitivity suggests that no single α works universally. Practitioners should use CKA-based pre-screening to identify layers where $\|h_B - h_A\|$ is small before applying high mixing ratios; the bound in Eq. (3) provides a direct diagnostic.

Theory Connection. Both failure modes are predicted by Theorem 1. L27 collapse corresponds to large $\|h_B - h_A\|$ at the final layer (high curvature K_g), which causes the correction terms to dominate the bound. Alpha sensitivity reflects the $\alpha(1-\alpha)$ factor: it peaks at α=0.5 but is amplified by large $\|h_B - h_A\|$, so the same α that is safe in well-aligned layers (L20–L25) becomes destructive in misaligned ones (L27). The theory thus not only explains the observed failures but provides a principled criterion for safe layer and ratio selection.

5.5 Cost Analysis

Latent merging incurs 1.96× latency overhead (5.52 s vs. 2.82 s per query on a single A100, batch size 1), with time-to-first-token doubling from 0.023 s to 0.044 s and decode throughput halving from 45.7 to 23.4 tokens/s. Peak VRAM remains essentially unchanged (29.1 GB; +18 MB). This overhead stems from two concurrent forward passes per token. Weight merging retains single-model cost after an offline merge, but sacrifices reversibility and dynamic control. Two directions can reduce latent merging overhead: sparse intervention at selected layers (e.g., L20–L25) and knowledge distillation [10] into a single model.

6 Limitations

Latent merging offers compelling advantages in controllability and reversibility, but several limitations warrant discussion.

Computational overhead. Latent merging requires two concurrent forward passes, incurring approximately 1.96× latency overhead compared to a single model. This makes it unsuitable for latency-critical or resource-constrained deployments.

Model scope. Our evaluation covers two model families (Qwen2.5 and Llama 3.1), both satisfying the shared-architecture prerequisite. Cross-architecture merging with different hidden dimensions or tokenizers requires learned projection layers, which we leave to future work.

Scale. Experiments are conducted at 7B–8B parameters. Feasibility at 70B+ is unclear due to memory requirements for concurrent forward passes.

7 Conclusion

We introduced Latent Merging, a framework that composes large language models by interpolating hidden representations rather than model parameters. By extending weight-space merging operators (LERP, SLERP, RegMean) to the latent domain, our approach enables dynamic, reversible, and layer-selective model composition without modifying weights.

Our theoretical analysis derives local second-order bounds under RMSNorm nonlinearity, yielding three actionable predictions: merge at later layers, use moderate-to-high mixing ratios ($\alpha \approx 0.5$), and align LM heads. Experiments on Qwen2.5-7B-Instruct and OpenThinker3-7B confirm these predictions: the best latent configuration (RegMean at L25, α=0.5) achieves 96.9% overall win-rate on JudgeBench versus 15.7% for the best weight-merging variant. On standard benchmarks, all three latent operators match or surpass both source models—Latent-LERP reaches 87.0% on HumanEval (vs. Qwen's 78.0%) and Latent-RegMean reaches 93.0% on GSM8K (vs. Qwen's 91.7%)—while weight merging collapses to 25.7% on GSM8K. Cross-family experiments on Llama-3.1-8B-Instruct and Nemotron-Nano confirm these findings generalize: latent merging achieves 67.1% pass@1 on HumanEval (vs. 58.5% for weight-SLERP) and 90.0% on GSM8K (vs. 88.0%), with latent-LERP reaching 75.9% JudgeBench win-rate compared to 67.8% for weight-LERP. Representational similarity analysis (CKA > 0.8) confirms that latent merging preserves semantic structure more effectively than weight merging.

We also identify and characterize failure modes: the final layer (L27) consistently collapses, and performance is sensitive to α in early layers. These findings, predicted by our theoretical bounds, provide practical guidance for deployment.

Future directions include: cross-architecture merging via learned projections, sparse layer selection to reduce inference overhead, knowledge distillation of merged behavior for single-model deployment, and scaling experiments beyond 7B–8B parameters.

References

1. Anonymous: TaLoS: Efficient and accurate TaLoRA merging via singular value decomposition. In: International Conference on Learning Representations (2025)
2. Bercovich, A., Dun, C., Joshi, D., et al.: Llama-nemotron: Efficient reasoning models. arXiv preprint arXiv:2505.00949 (2025)
3. Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.d.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)
4. Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., Schulman, J.: Training verifiers to solve math word problems. arXiv preprint (2021)
5. Dathathri, S., Madotto, A., Lan, J., Hung, J., Frank, E., Molino, P., Yosinski, J., Liu, R.: Plug and play language models: A simple approach to controlled text generation. In: International Conference on Learning Representations (ICLR) (2020), `https://openreview.net/forum?id=H1edEyBKDS`
6. Diallo, K.K.S.J.D., et al.: Style vectors for steering generative large language models. In: Findings of EACL (2024), `https://aclanthology.org/anthology-files/pdf/findings/2024.findings-eacl.52.pdf`
7. Fedus, W., Zoph, B., Shazeer, N.: Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. Journal of Machine Learning Research **23**(120), 1–39 (2022)

8. Grattafiori, A., Dubey, A., Jauhri, A., et al.: The llama 3 herd of models. arXiv preprint arXiv:2407.21783 (2024)
9. Guha, E., et al.: Openthinker-3: Democratizing reasoning with adaptive thinking. arXiv preprint (2025)
10. Hinton, G., Vinyals, O., Dean, J.: Distilling the knowledge in a neural network. arXiv preprint (2015)
11. Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Dang, K., et al.: Qwen2.5: A party of foundation models. arXiv preprint arXiv:2412.15115 (2024)
12. Ilharco, G., Ribeiro, M.T., Wortsman, M., Gururangan, S., Schmidt, L., Hajishirzi, H., Farhadi, A.: Editing models with task arithmetic. In: International Conference on Learning Representations (ICLR) (2023), https://arxiv.org/abs/2212.04089
13. Jin, X., Ren, X., Preotiuc-Pietro, D., Cheng, P.: Dataless knowledge fusion by merging weights of language models. In: International Conference on Learning Representations (2023)
14. Kornblith, S., Norouzi, M., Lee, H., Hinton, G.: Similarity of neural network representations revisited. In: Proceedings of the 36th International Conference on Machine Learning (ICML) (2019), https://proceedings.mlr.press/v97/kornblith19a.html
15. Matena, M.S., Raffel, C.: Merging models with fisher-weighted averaging. In: Advances in Neural Information Processing Systems (2022), https://arxiv.org/abs/2111.09832
16. Panickssery, N., Gabrieli, N., Schulz, J., Tong, M., Hubinger, E., Turner, A.: Steering llama 2 via contrastive activation addition. arXiv preprint (2023), https://arxiv.org/abs/2312.06681
17. Panigrahi, A., et al.: Localizing task information in the weights of a pretrained language model. Transactions on Machine Learning Research (2024)
18. Shi, P., et al.: Judging the judges: A systematic study of position bias in llm-as-a-judge. arXiv preprint (2025)
19. Subramani, N., Suresh, N., Peters, M.E.: Extracting latent steering vectors from pretrained language models. In: Findings of the Association for Computational Linguistics (ACL) (2022), https://aclanthology.org/2022.findings-acl.48
20. Turner, A.M., Thiergart, L., Leech, G., Udell, D., MacDiarmid, M., Belrose, N., et al.: Steering language models with activation engineering. arXiv preprint (2023), https://arxiv.org/abs/2308.10248
21. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems (2017), https://arxiv.org/abs/1706.03762
22. Wortsman, M., Ilharco, G., Gadre, S., Roelofs, R., Gontijo-Lopes, R., Morcos, A., Namkoong, H., Farhadi, A., Carmon, Y., Kornblith, S., Schmidt, L.: Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In: International Conference on Machine Learning (ICML) (2022), https://arxiv.org/abs/2203.05482
23. Yadav, P., Tam, D., Choshen, L., Raffel, C., Bansal, M.: Ties-merging: Resolving interference when merging models. arXiv preprint (2023), https://arxiv.org/abs/2306.01708
24. Zhang, B., Sennrich, R.: Root mean square layer normalization. Advances in Neural Information Processing Systems (2019)